LAPORAN PRAKTIKUM
PEMROGRAMAN BERORIENTASI OBJEK
MODUL [7] – Abstract dan Interface



Rimbhi Putri Aulia Azzahra

103132400018

PS1DS-05-REG-01

PROGRAM STUDI SAINS DATA

FAKULTAS INFORMATIKA

TELKOM UNIVERSITY PURWOKERTO

2026

A. Dasar Teori

- Abstract Class: kelas yang objeknya tidak dapat dibuat secara langsung dan digunakan sebagai dasar bagi kelas turunannya.
- Inheritance: konsep pewarisan atribut dan metode dari kelas induk kepada kelas turunan menggunakan keyword extends.
- Interface: kontrak yang berisi deklarasi metode dan harus diimplementasikan oleh kelas menggunakan keyword implements.
- Enkapsulasi: untuk melindungi atribut dengan access modifier private dan menyediakan getter untuk mengakses data.
- Polimorfisme: dimana objek dari kelas berbeda diproses untuk melalu tipe referensi yang sama.
- Method Overriding: proses mendefinisikan ulang metode dari kelas induk pada kelas turunan sesuai kebutuhan.
- Default dan Static method: metode interface yang memiliki implementasi sedangkan static method dapat dipanggil langsung melalui nama interface tanpa membuat objek.

B. Guided

1. Guided01
    - Employee.java

```java
package com.rimbhi.guided.guided01;

public abstract class Employee {
    private String name;

    public Employee(String name){
        this.name = name;
    }

    public String getName(){
        return name;
    }

    // setiap pegawai pasti punya gaji, tetapi dengan rumus yang berbeda-beda.
    public abstract double earnings();
}
```

    - SalariedEmployee.java

```java
package com.rimbhi.guided.guided01;
```

```java
public class SalariedEmployee extends Employee {
    private double weeklySalary;

    public SalariedEmployee(String name, double
weeklySalary){
        super(name);
        this.weeklySalary = weeklySalary;
    }

    @Override
    public double earnings(){
        return weeklySalary;
    }

}
```

- Main.java

```java
package com.rimbhi.guided.guided01;

public class Main {
    public static void main(String[] args) {
        SalariedEmployee salaried = new
SalariedEmployee("Budi", 5000000);
        System.out.println("Nama: " + salaried.getName());
        System.out.println("Gaji: Rp" +
salaried.earnings());

    }
}
```

Output:

```
Nama: Budi
Gaji: Rp5000000.0
```

2. Guided02
    - Employee.java

```java
package com.rimbhi.guided.guided02;

public abstract class Employee implements Payable{
    private String name;

    public Employee(String name){
        this.name = name;
    }

    public String getName(){
        return name;
```

```
    }

    // setiap pegawai pasti punya gaji, tetapi dengan rumus
yang berbeda-beda.
    public abstract double earnings();

    // saat ada penagihan, alihkan langsung ke metode
kalkulasi gaji (earning)
    @Override
    public double getPaymentAmount(){
        return earnings();
    }
}
```

- Payable.java

```
package com.rimbhi.guided.guided02;

public interface Payable {
    double getPaymentAmount();
}
```

- Invoice.java

```
package com.rimbhi.guided.guided02;

public class Invoice implements Payable {
    private String partName;
    private int quantity;
    private double pricePerItem;

    public Invoice(String partName, int quantity, double
pricePerItem){
        this.partName = partName;
        this.quantity = quantity;
        this.pricePerItem = pricePerItem;
    }

    public String getPartName(){
        return partName;
    }

    @Override
    public double getPaymentAmount(){
        return quantity * pricePerItem;
    }

}
```

- SalariedEmployee.java

```
package com.rimbhi.guided.guided02;

public class SalariedEmployee extends Employee {
     private double weeklySalary;

    public SalariedEmployee(String name, double
weeklySalary){
        super(name);
        this.weeklySalary = weeklySalary;
    }

    @Override
    public double earnings(){
        return weeklySalary;
    }

}
```

- Main.java

```
package com.rimbhi.guided.guided02;

public class Main {
    public static void main(String[] args) {
        Payable[] items = new Payable[2];
        items[0] = new Invoice("Monitor LED", 2, 1500000);
        items[1] = new SalariedEmployee("Budi", 5000000);

        // Dynamic dispatch : java otomatis mengeksekusi
rumus yang tepat sesuai objek

        for (Payable item: items){
            System.out.println("jumlah yang harus dibayar:
Rp" + item.getPaymentAmount());
        }
    }
}
```

Output:

```
jumlah yang harus dibayar: Rp3000000.0
jumlah yang harus dibayar: Rp5000000.0
```

3. Guided03
    - Employee.java

```
package com.rimbhi.guided.guided03;

public abstract class Employee implements Payable{
```

```
     private String name;

    public Employee(String name){
        this.name = name;
    }

    public String getName(){
        return name;
    }

    // setiap pegawai pasti punya gaji, tetapi dengan rumus
yang berbeda-beda.
    public abstract double earnings();

    // saat ada penagihan, alihkan langsung ke metode
kalkulasi gaji (earning)
    @Override
     public double getPaymentAmount(){
        return earnings();
    }
}
```

- Payable.java

```
package com.rimbhi.guided.guided03;

public interface Payable {
    double getPaymentAmount();

    default void printReceipt(){
        System.out.println("Struk Resmi - Total: Rp " +
getPaymentAmount());
    }

    // fungsi utilitas
    static void logTransaction(String message){
        System.out.println("[AUDIT LOG]" + message);
    }
}
```

- Invoice.java

```
package com.rimbhi.guided.guided03;

public class Invoice implements Payable {
    private String partName;
    private int quantity;
    private double pricePerItem;
```

```
    public Invoice(String partName, int quantity, double
pricePerItem){
        this.partName = partName;
        this.quantity = quantity;
        this.pricePerItem = pricePerItem;
    }

    public String getPartName(){
        return partName;
    }

    @Override
    public double getPaymentAmount(){
        return quantity * pricePerItem;
    }

}
```

- SalariedEmployee.java

```
package com.rimbhi.guided.guided03;

public class SalariedEmployee extends Employee {
     private double weeklySalary;

    public SalariedEmployee(String name, double
weeklySalary){
        super(name);
        this.weeklySalary = weeklySalary;
    }

    @Override
    public double earnings(){
        return weeklySalary;
    }

}
```

- Main.java

```
package com.rimbhi.guided.guided03;

public class Main {
    public static void main(String[] args) {
        Payable.logTransaction("Sistem pembayaran
diinisialisasi.");

        Payable[] items = new Payable[2];
```

```
        items[0] = new Invoice("Ram DDR 5 16GB", 2,
2000000);
        items[1] = new SalariedEmployee("Budi", 5000000);

        for (Payable item : items){
            item.printReceipt();
        }

        Payable.logTransaction("Transaksi selesai
diproses.");


    }
}
```

Output:

```
[AUDIT LOG]Sistem pembayaran diinisialisasi.
Struk Resmi - Total: Rp 4000000.0
Struk Resmi - Total: Rp 5000000.0
[AUDIT LOG]Transaksi selesai diproses.
```

C. Unguided

1. Buatlah program dengan studi kasus sitem kasir bioskop (Cinema POS). sistem dirancang untuk memproses berbagai jenis transaksi, yaitu pembelian tiket bioskop reguler, tiket VIP, serta paket makanan dan minuman. Program harus menerapkan prinsip enkapsulasi, penggunaan constructor dengan keyword this, penggunaan super() pada kelas turunan, serta menggunakan array biasa tanpa ArrayList.

- Transaksi.java (Interface)

```
package com.rimbhi.unguided;

public interface ItemTransaksi {
   double getNominalBayar();
   String getNamaTransaksi();

   default void cetakRincianItem(){
    System.out.println("- " + getNamaTransaksi() + " | Total:
Rp " + getNominalBayar());
   }

   static void catatLogKasir(String pesan){
    System.out.println("[LOG SISTEM KASIR] " + pesan);
   }
}
```

- TiketBioskop.java

```java
package com.rimbhi.unguided;

public abstract class TiketBioskop implements ItemTransaksi{
    private String judulFilm;
    private double hargaDasar;
    private int jumlahKursi;

    public TiketBioskop(String judulFilm, double hargaDasar,
int jumlahKursi){
        this.judulFilm = judulFilm;
        this.hargaDasar = hargaDasar;
        this.jumlahKursi = jumlahKursi;
    }

    public String getJudulFilm(){
        return judulFilm;
    }

    public double getHargaDasar(){
        return hargaDasar;
    }

    public int getJumlahKursi(){
        return jumlahKursi;
    }

    @Override
    public double getNominalBayar(){
        return hitungTotalBayar();
    }

    public abstract double hitungTotalBayar();

    @Override
    public abstract String getNamaTransaksi();
}
```

- TiketReguler.java

```java
package com.rimbhi.unguided;

public class TiketReguler extends TiketBioskop{
    private int nomorStudio;

    public TiketReguler(String judulFilm, double hargaDasar,
int jumlahKursi, int nomorStudio){
        super(judulFilm, hargaDasar, jumlahKursi);
        this.nomorStudio = nomorStudio;
    }
```

```
    public int getNomorStudio(){
        return nomorStudio;
    }

    @Override
    public double hitungTotalBayar(){
        return getHargaDasar() * getJumlahKursi();
    }

    @Override
     public String getNamaTransaksi() {
        return "Tiket Reguler (Studio " + nomorStudio + ") - "
+ getJudulFilm() + " (" + getJumlahKursi() + " x Rp" +
getHargaDasar() + ")";
    }

}
```

- TiketVIP.java

```
package com.rimbhi.unguided;

public class TiketVIP extends TiketBioskop {
     private double biayaFasilitasVIP;

    public TiketVIP(String judulFilm, double hargaDasar, int
jumlahKursi, double biayaFasilitasVIP) {
        super(judulFilm, hargaDasar, jumlahKursi);
        this.biayaFasilitasVIP = biayaFasilitasVIP;
    }

    public double getBiayaFasilitasVIP() {
        return biayaFasilitasVIP;
    }

    @Override
    public double hitungTotalBayar() {
        return (getHargaDasar() + biayaFasilitasVIP) *
getJumlahKursi();
    }

    @Override
    public String getNamaTransaksi() {
        return "Tiket VIP Premiere - " + getJudulFilm() + " ("
+ getJumlahKursi() + " x Rp" + (getHargaDasar() +
biayaFasilitasVIP) + ")";
    }
```

```
}
```

- PaketSnack.java

```
package com.rimbhi.unguided;

public class PaketSnack implements ItemTransaksi{
    private String namaPaket;
    private int jumlahPorsi;
    private double hargaPerPorsi;

    public PaketSnack(String namaPaket, int jumlahPorsi, double
hargaPerPorsi) {
        this.namaPaket = namaPaket;
        this.jumlahPorsi = jumlahPorsi;
        this.hargaPerPorsi = hargaPerPorsi;
    }

    public String getNamaPaket() {
        return namaPaket;
    }

    public int getJumlahPorsi() {
        return jumlahPorsi;
    }

    public double getHargaPerPorsi() {
        return hargaPerPorsi;
    }

    @Override
    public double getNominalBayar() {
        return jumlahPorsi * hargaPerPorsi;
    }

    @Override
    public String getNamaTransaksi() {
        return "F&B Snack: " + namaPaket + " (" + jumlahPorsi +
" x Rp" + hargaPerPorsi + ")";
    }
}
```

- Main.java

```
package com.rimbhi.unguided;

public class Main {
    public static void main(String[] args) {
```

```
        ItemTransaksi.catatLogKasir("Membuka transaksi baru
untuk Pelanggan.");

        TiketReguler tiketReguler = new
TiketReguler("Interstellar", 40000, 2, 3);

        TiketVIP tiketVIP = new TiketVIP("Dune: Part Two",
60000, 2, 25000);

        PaketSnack paketSnack = new PaketSnack("Combo Popcorn
Caramel + 2 Soda", 1, 55000);

        ItemTransaksi[] daftarTransaksi = {
            tiketReguler,
            tiketVIP,
            paketSnack
        };

        double grandTotal = 0;

        System.out.println("=== DAFTAR TRANSAKSI BIOSKOP ===");

        for (ItemTransaksi item : daftarTransaksi) {
            item.cetakRincianItem();
            grandTotal += item.getNominalBayar();
        }

        System.out.println("----------------------------------
-------------------------");
        System.out.println("TOTAL TRANSAKSI : Rp" +
grandTotal);
        System.out.println("==================================
=========================");

        ItemTransaksi.catatLogKasir("Transaksi selesai
diproses.");
    }
}
```

Penjelasan:

a. ItemTransaksi.java: merupakan interface yang fungsinya untuk dijadikan kontrak setiap objek transaksi dalam sistem kasir. Terdapat dua mentode utama, getNominalBayar() untuk ambil total pembayaran dan getNamaTransaksi() untuk ambil nama transaksi. Terdapat juga default method cetakRincianItem() digunakan untuk menampilkan rincian transaksi secara otomatis. Ada static method catatLogKasir() untuk menampilkan aktivitas kasir.

b. TiketBioskop.java: kelas abstract yang menjadi induk bagi kelas TiketReguler dan TiketVIP, terdapat tiga atribtu ada judulFilm, hargaDasar, dan jumlahKursi yang akses modifiernya private. Terdapat 2 metode abstrak yaitu hitungTotalBayar() dan getNamaTransaksi(). Yang implementasinya diserahkan di kelas keturunannya. Kelas ini juga mengimplementasi interface ItemTransaksi dengan membuat metode fetNominalBayar() yang mengembalikan hasil dari hitungTotalBayar().
c. TiketReguler.java: kelas ini adalah turunan dari TiketBioskop yang digunakan untuk mengelola pembelian tiket bioskop reguler. Dengan atribut tambhan berupa nomorStudio, digunakan juga keyword super untuk memanggil constructor kelas induk dan menginisialisasikan atribut yang diwariskan. Metode hitungTotalBayar() di override untuk menghitung total pembayaran berdasarkan harga dasar dikalikan jumlah kuris. Metode getNamaTransaksi() digunakan untuk menghasilkan informasi tiket reguler yang nantinya ditampilkan pada struk transaksi.
d. TiketVIP.java: kelas turunan dari TiketBioskop yang digunakan untuk mengelola pembelian tiket VIP. Kelas ini memiliki atribut tambahan bernama biayaFasilitasVIP, yaitu biaya tambahan untuk fasilitas khusus pada setiap kursi.
e. Perhitungan total pembayaran dilakukan dengan menjumlahkan harga dasar dan biaya fasilitas VIP, kemudian dikalikan dengan jumlah kursi. Metode getNamaTransaksi() juga di override untuk menampilkan informasi pembelian tiket VIP beserta harga yang sudah termasuk biaya fasilitas.
f. PaketSnack.java: kelas independen yang digunakan untuk mengelola transaksi pembelian makanan dan minuman di bioskop. Kelas ini tidak memiliki hubungan pewarisan dengan TiketBioskop, tetapi tetap dapat diproses oleh sistem kasir karena mengimplementasikan interface ItemTransaksi. Kelas ini memiliki tiga atribut, yaitu namaPaket, jumlahPorsi, dan hargaPerPorsi. Metode getNominalBayar() digunakan untuk menghitung total pembayaran berdasarkan jumlah porsi dikalikan harga per porsi. Sementara itu, metode getNamaTransaksi() digunakan untuk menampilkan informasi paket snack yang dibeli.
g. Main.java: program memanggil metode catatLogKasir() untuk menampilkan informasi bahwa transaksi baru telah dibuka. Setelah itu, dibuat tiga objek transaksi, yaitu TiketReguler, TiketVIP, dan PaketSnack. Ketiga objek tersebut kemudian dimasukkan ke dalam satu array bertipe ItemTransaksi[]. Penggunaan array ini menunjukkan penerapan polimorfisme karena objek dari kelas yang berbeda dapat diproses melalui satu tipe interface yang sama. Program menggunakan perulangan

for-each untuk memanggil metode cetakRincianItem() dan mengambil nominal pembayaran dari setiap objek. Nilai pembayaran kemudian dijumlahkan secara bertahap menggunakan variabel grandTotal.

Output:

```
[LOG SISTEM KASIR] Membuka transaksi baru untuk Pelanggan.
=== DAFTAR TRANSAKSI BIOSKOP ===
- Tiket Reguler (Studio 3) - Interstellar (2 x Rp40000.0) | Total: Rp 80000.0
- Tiket VIP Premiere - Dune: Part Two (2 x Rp85000.0) | Total: Rp 170000.0
- F&B Snack: Combo Popcorn Caramel + 2 Soda (1 x Rp55000.0) | Total: Rp 55000.0
------------------------------------------------------------
TOTAL TRANSAKSI : Rp305000.0
============================================================
[LOG SISTEM KASIR] Transaksi selesai diproses.
```

D. Kesimpulan

konsep abstract class digunakan untuk membentuk kelas induk yang tidak dapat diinstansiasi secara langsung, sedangkan interface digunakan sebagai kontrak agar objek dari kelas yang berbeda dapat memiliki standar metode yang sama.